

Supplementary Material

Supplementary Material

Companion to the paper “Evolvable Decision Programs: Explainable Page Composition under Production Reward Conditions”. Contains reproduction commands, code structure, and instructions for the interactive demos. Numerical and methodological content stays in the main paper.

A. Reproducing the experiments

All experiments share a 10K-session stream at seed 42. Bandit reps vary the policy seed; EDP-canned and EDP-static are deterministic; EDP-agent and EDP-OPRO replicates are independent subagent draws. Resource budget per replicate: ~30 s for a bandit run, ~20 s of subagent latency per agent edit checkpoint.

A.1 Multi-seed baselines

```
# parametric persona source (paper §2.1)
python3 experiments/multiseed.py --reps 10 \
    --out results_multiseed_parametric_cat.npz

# LLM persona source (paper §2.2c)
python3 experiments/persona_generate.py prepare      # writes per-persona
instruction files
# spawn 14 subagents, one per file in data/persona_instructions/, each
# writing data/persona_drafts/<name>.json
python3 experiments/persona_generate.py merge        # consolidates into cache
EDP_PERSONA_SOURCE=llm python3 experiments/multiseed.py --reps 10 \
    --source llm --out results_multiseed_llm_cat.npz
```

A.2 §5.8 baselines (static, LLM-as-policy, Robust EDP)

```
# static + LLM-as-policy (deterministic; one subagent call to author the policy)
python3 experiments/llm_policy_generate.py           # writes the prompt
# spawn 1 subagent → data/llm_policy_fn.py
python3 experiments/run_baselines.py --source llm    # runs both deterministic
baselines

# Robust EDP: 3 rounds, one subagent per round
EDP_PERSONA_SOURCE=llm python3 -m edp.orchestrators.robust \
    --reset --state-dir robust_state_llm --until 2500
# spawn subagent → robust_state_llm/edits_round_2500.json
EDP_PERSONA_SOURCE=llm python3 -m edp.orchestrators.robust \
    --state-dir robust_state_llm \
    --apply robust_state_llm/edits_round_2500.json --until 5000
# repeat for 7500 and 10000
```

A.3 Stressor and lab-vs-real ablations

```
python3 experiments/stressor_decomp.py      # §5.2 – bandit cum regret per condition
python3 experiments/lab_vs_real.py --reps 10 # §5.9 – lab vs production for both
sources
```

A.4 Figures

```
python3 viz.py                               # all figures into figures/
python3 experiments/build_icml_pdf.py         # builds paper-icml.pdf
```

B. Code structure

```
edp/                                     # core package
├─ config.py                           # constants: N_SLOTS, signal/need/problem names
├─ catalog.py                          # widget catalog + fashion categories
├─ ground_truth.py                     # category-aware reward, oracle, persona-source
switch
├─ sim.py                             # (persona, category, feat) stream +
DelayedFeedback
├─ personas/
│   └─ parametric.py                  # paper §2.1 baseline (8 personas, fixed
Gaussians)
│   └─ llm.py                         # cache reader for LLM-generated personas
├─ policies/
│   └─ base.py                        # Policy ABC
│   └─ edp.py                        # EDPPolicy + apply_edits
│   └─ bandit.py                     # LinTS + warm/cold/category contexts
│   └─ static.py                     # §5.8 static-widget baseline (top-6 by base)
│   └─ llm_policy.py                 # §5.8 LLM-as-policy wrapper for data/
llm_policy_fn.py
├─ orchestrators/
│   └─ base.py                        # state save/load, batch run
│   └─ report.py                     # report-based live-agent loop
│   └─ opro.py                       # OPRO ablation loop
│   └─ robust.py                     # §5.8 Robust EDP – K-perturbation validation-
slice selection

experiments/                           # entry-point scripts (paper-aligned)
├─ compare.py                         # head-to-head harness
├─ multiseed.py                       # 10-seed bandits + deterministic EDP baselines
├─ persona_generate.py                # prepare/merge pipeline for LLM persona cache
├─ llm_policy_generate.py             # §5.8 prompt for the LLM-as-policy function
├─ run_baselines.py                  # §5.8 static + LLM-as-policy on either source
├─ stressor_decomp.py                # §5.2 sweep
├─ lab_vs_real.py                    # §5.9 lab vs production
├─ export_lints_state.py              # train LinTS, export weights JSON for the demo
└─ build_icml_pdf.py                 # builds the ICML two-column PDF

data/
├─ personas_text.yaml                # 14 persona descriptions (NL)
├─ persona_instructions/             # per-persona subagent prompts
├─ persona_drafts/                   # subagent outputs
├─ personas_llm_cache.json           # consolidated cache
├─ widget_descriptions.yaml          # widget descriptions for the §5.8 LLM-policy
prompt
└─ llm_policy_fn.py                  # §5.8 LLM-authored pick_page(feat, category)

demo/
├─ policy_comparison.html             # single-file HTML interactive companions
├─ edp_orchestrator.html             # §5 head-to-head: 5 policies side by side
├─ lints_state.json                  # §3 composition-flow visualiser
└─                               # pre-trained LinTS weights for the demo
```

The Policy ABC (`edp/policies/base.py`) is the integration point for new methods. Adding a new policy means writing a `select_page(feat) -> list[str]` (and optionally `record_feedback(payload, observed_reward)` for online learners) and wiring it into `experiments/compare.py` and `experiments/multiseed.py`.

The persona-source interface (`edp/personas/`) is the integration point for new simulators. Both sources expose (`persona_names`, `mixture_weights`, `true_needs`, `sample_session`); switching between them is `set_source('llm')` or env var `EDP_PERSONA_SOURCE=llm`.

C. Interactive demos

Two single-file HTML pages under `demo/` (no build step; Tailwind via CDN):

- `policy_comparison.html` — §5 companion. Five side-by-side policy panels (Static, LinTS-cold, LinTS-warm, EDP-v1, EDP-v3) on the same session. Pick persona + fashion category, drag the 14 signal sliders, see each policy's 6-slot page and its reward / % of oracle. LinTS panels use pre-trained posterior means exported from a 10K-session Python training run (`demo/lints_state.json`, ~784 KB, regenerable with `experiments/export_lints_state.py`). The JS reward function matches the Python implementation exactly (verified on `returner_anxious × bottoms`: Static 1.101125, EDP-v1 1.042475).
- `edp_orchestrator.html` — §3 companion. EDP composition flow, Layer-1 PWL shapes → 7-d problem fingerprint → Layer-2 GAM scoring → greedy composition, with a v1 ↔ v3 toggle for the canned edit batch.

Open with:

```
cd demo && python3 -m http.server 8765
# open http://localhost:8765/policy_comparison.html
```

The HTTP server is needed for `policy_comparison.html` because it fetches `lints_state.json` (file:// origin would be blocked). The composition-flow demo works via `file://` directly.

D. Hyperparameters

LinTS: - exploration scale $\alpha = 0.3$ - prior precision $\lambda = 1.0$ - per-slot Cholesky-based Thompson sample - masked actions (no widget repeats within a page)

EDP: - 6 slots, 22 widgets, 7 latent problems - Greedy submodular composition (no backtracking) - LLM agent edit batches limited to 8–16 atomic edits per checkpoint, magnitude range -1.5 to 2.8

Production reward stack (default): - delay = 500 sessions (~2 days at 250 sessions/day) - noise $\sigma = 0.20$ added on release from delay queue - page-level attribution: each slot receives `page_total / N_SLOTS` as its credit

Robust EDP wrapper (§5.8): - $K = 8$ perturbations per checkpoint - perturbation $\sigma = 0.15$ (Gaussian) on every parameter the agent's edits touched, except `slot_decay` - validation slice = 500 sessions at seed 99991 - selection score = mean $- 0.5 \cdot \text{std}$ of validation reward